EJERCICIOS RESUELTOS CLASES ANÓNIMAS Y EXPRESIONES LAMBDA:

1. Crear una función anónima que nos permita calcular el promedio de un arreglo de números enteros.

Con Clases Anónimas:

```
import java.util.Arrays;

public class Promedio {
    public static void main(String[] args) {
        int[] numeros = {5, 10, 15, 20, 25};
        double promedio = calcularPromedio(numeros);
        System.out.println("Promedio: " + promedio);
    }

    static double calcularPromedio(int[] numeros) {
        Operacion promedioOperacion = new Operacion() {
            @Override
            public double realizarOperacion(int[] numeros) {
                return Arrays.stream(numeros).average().orElse(0);
            }
        };
        return promedioOperacion.realizarOperacion(numeros);
    }
```

Con Expresiones Lambda:

```
import java.util.Arrays;

public class EjerciciosLambda {
    public static void main(String[] args) {
        int[] numeros = {5, 8, 12, 3, 7};

        // Expresión lambda para calcular el promedio
        CalculadoraPromedio calculadoraPromedio = array ->
Arrays.stream(array).average().orElse(0.0);

        double promedio =
calculadoraPromedio.calcularPromedio(numeros);
        System.out.println("Promedio: " + promedio);
    }

    interface CalculadoraPromedio {
        double calcularPromedio(int[] array);
    }
}
```

2. Crear una función anónima que calcule el factorial dado un número entero.

Con Clases Anónimas:

```
public class Factorial {
    public static void main(String[] args) {
        int numero = 5;
        long resultado = calcularFactorial(numero);
        System.out.println("Factorial de " + numero + ": " +
resultado);
    }

    static long calcularFactorial(int numero) {
        OperacionFactorial factorialOperacion = new
OperacionFactorial() {
            @Override
            public long calcular(int numero) {
                return numero == 0 ? 1 : LongStream.rangeClosed(1,
numero).reduce(1, (x, y) -> x * y);
            }
        };
        return factorialOperacion.calcular(numero);
    }

    interface OperacionFactorial {
        long calcular(int numero);
    }
}
```

Con Expresiones Lambda:

```
public class EjerciciosLambda {
    public static void main(String[] args) {
        int numero = 5;

        // Expresión lambda para calcular el factorial
        CalculadoraFactorial calculadoraFactorial = n -> n == 0 ? 1 :
n * calculadoraFactorial.calcularFactorial(n - 1);

        int factorial =
calculadoraFactorial.calcularFactorial(numero);
        System.out.println("Factorial de " + numero + ": " +
factorial);
    }

    interface CalculadoraFactorial {
        int calcularFactorial(int n);
    }
}
```

3. Crear una función anónima que permita conocer si un número es par.

Con Clases Anónimas:

```
public class NumeroPar {
    public static void main(String[] args) {
        int numero = 8;
        boolean esPar = verificarPar(numero);
        System.out.println(numero + " es par: " + esPar);
    }

    static boolean verificarPar(int numero) {
        VerificacionPar verificacionPar = new VerificacionPar() {
            @Override
            public boolean esPar(int numero) {
                return numero % 2 == 0;
            }
        };
        return verificacionPar.esPar(numero);
    }

    interface VerificacionPar {
        boolean esPar(int numero);
    }
}
```

Con Expresiones Lambda:

```
public class EjerciciosLambda {
    public static void main(String[] args) {
        int numero = 7;

        // Expresión lambda para verificar si un número es par
        VerificadorPar verificadorPar = n -> n % 2 == 0;

        boolean esPar = verificadorPar.verificarPar(numero);
        System.out.println("¿Es par? " + esPar);
    }

    interface VerificadorPar {
        boolean verificarPar(int n);
    }
}
```

4. Dado un arreglo de números enteros, crear una función anónima que retorne el número mayor.

Con Clases Anónimas:

```
import java.util.Arrays;

public class NumeroMayor {
    public static void main(String[] args) {
        int[] numeros = {15, 8, 20, 12, 25};
        int mayor = obtenerNumeroMayor(numeros);
        System.out.println("Número mayor: " + mayor);
    }

    static int obtenerNumeroMayor(int[] numeros) {
        ObtencionNumeroMayor numeroMayorOperacion = new
ObtencionNumeroMayor() {
            @Override
            public int obtener(int[] numeros) {
                return Arrays.stream(numeros).max().orElse(0);
            }
        };
        return numeroMayorOperacion.obtener(numeros);
    }

    interface ObtencionNumeroMayor {
        int obtener(int[] numeros);
    }
}
```

Con Expresiones Lambda:

```
import java.util.Arrays;

public class EjerciciosLambda {
    public static void main(String[] args) {
        int[] numeros = {14, 8, 22, 5, 17};

        // Expresión lambda para encontrar el número mayor
        EncontrarMayor encontrarMayor = array ->
Arrays.stream(array).max().orElse(0);

        int numeroMayor = encontrarMayor.encontrarMayor(numeros);
        System.out.println("Número Mayor: " + numeroMayor);
    }

    interface EncontrarMayor {
        int encontrarMayor(int[] array);
    }
}
```

5. Dado un arreglo de números enteros, crear una función anónima que retorne el número menor.

Con Clases Anónimas:

```
import java.util.Arrays;

public class NumeroMenor {
    public static void main(String[] args) {
        int[] numeros = {15, 8, 20, 12, 25};
        int menor = obtenerNumeroMenor(numeros);
        System.out.println("Número menor: " + menor);
    }

    static int obtenerNumeroMenor(int[] numeros) {
        ObtencionNumeroMenor numeroMenorOperacion = new
ObtencionNumeroMenor() {
            @Override
            public int obtener(int[] numeros) {
                return Arrays.stream(numeros).min().orElse(0);
            }
        };
        return numeroMenorOperacion.obtener(numeros);
    }

    interface ObtencionNumeroMenor {
        int obtener(int[] numeros);
    }
}
```

Con Expresions Lambda:

```
import java.util.Arrays;

public class EjerciciosLambda {
    public static void main(String[] args) {
        int[] numeros = {14, 8, 22, 5, 17};

        // Expresión lambda para encontrar el número menor
        EncontrarMenor encontrarMenor = array ->
Arrays.stream(array).min().orElse(0);

        int numeroMenor = encontrarMenor.encontrarMenor(numeros);
        System.out.println("Número Menor: " + numeroMenor);
    }

    interface EncontrarMenor {
        int encontrarMenor(int[] array);
    }
}
```

6. Dado un arreglo de números enteros, crear una función anónima que retorne el número que más se repite

Con Clases Anónimas:

```
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        int[] numeros = {1,2,3,4,5,6,7,8,9,10,2,2,3,3,3};
        // Clase anónima para encontrar el número que más se repite
        int numeroMasRepetido = new Object() {
            int encontrarNumeroMasRepetido(int[] nums) {
                Map<Integer, Integer> frecuenciaNumeros = new HashMap<>();
                int maxFrecuencia = 0;
                int numMasRepetido = 0;

                // Contar la frecuencia de cada número en el arreglo
                for (int num : nums) {
                    int frec = frecuenciaNumeros.getOrDefault(num, 0) + 1;
                    frecuenciaNumeros.put(num, frec);
                    if (frec > maxFrecuencia) {
                        maxFrecuencia = frec;
                        numMasRepetido = num;
                    }
                }
                return numMasRepetido;
            }
        }.encontrarNumeroMasRepetido(numeros);

        System.out.println("El número más repetido: " + numeroMasRepetido);
    }
}
```

Con Expresiones Lambda:

```
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        int[] numeros = {1, 2, 3, 4, 4, 5, 6, 6, 6, 7};

        int numMasRepetido = encontrarNumeroMasRepetido(numeros);
        System.out.println("Número que más se repite: " +
numMasRepetido);
    }
    public static int encontrarNumeroMasRepetido(int[] numeros) {
        Map<Integer, Integer> conteo = new HashMap<>();

        Arrays.stream(numeros).forEach(num -> conteo.put(num,
conteo.getOrDefault(num, 0) + 1));
        return conteo.entrySet().stream()
            .max(Map.Entry.comparingByValue())
            .map(Map.Entry::getKey)
            .orElse(-1);
    }
}
```

7. Crear una función anónima que reciba como parámetro 3 numeros enteros. La función retorna el número mayor.

Con Clases Anónimas

```
public class Main {
    public static void main(String[] args) {
        // Define los tres números enteros
        int num1 = 15;
        int num2 = 27;
        int num3 = 10;

       // Crea una instancia de la clase anónima
        FuncionMayor mayor = new FuncionMayor() {
            @Override
            public int encontrarMayor(int a, int b, int c) {
                // Encuentra el mayor de los tres números
                return Math.max(Math.max(a, b), c);
            }
        };

        // Llama al método de la clase anónima
        int mayorNumero = mayor.encontrarMayor(num1, num2, num3);
        System.out.println("El mayor número es: " + mayorNumero);
    }

    // Interfaz funcional.
    interface FuncionMayor {
        int encontrarMayor(int a, int b, int c);
    }
}
```

Con Expresiones Lambda:

```
public class EjerciciosLambda {

    interface EncontrarMayorEntreTres {
        int encontrarMayorEntreTres(int a, int b, int c);
    }

    public static void main(String[] args) {
        int num1 = 15, num2 = 8, num3 = 20;

// Expresión lambda para encontrar el mayor de tres números
        EncontrarMayorEntreTres encontrarMayorEntreTres = (a, b, c) -
> Math.max(Math.max(a, b), c);

        int mayorEntreTres =
encontrarMayorEntreTres.encontrarMayorEntreTres(num1, num2, num3);

        System.out.println("Número Mayor entre Tres: " +
mayorEntreTres);
    }

}
```

8. Crear una función anónima que reciba dos parámetros, un string y un numero entero. La función retorna un string el cual será el resultado de multiplicar ambos parámetros.

Con Clases Anónimas:

```
public class Main {
    public static void main(String[] args) {
        String resultado = multiplicar("Hola", 3);
        System.out.println("Resultado de la multiplicación: " + resultado);
    }

    public static String multiplicar(final String cadena, final int num) {
        return new Operacion<String, Integer>() {
            @Override
            public String ejecutar(String cadena, Integer num) {
                StringBuilder resultado = new StringBuilder();
                for (int i = 0; i < num; i++) {
                    resultado.append(cadena);
                }
                return resultado.toString();
            }
        }.ejecutar(cadena, num);
    }
}

interface Operacion<T, U> {
    T ejecutar(T t, U u);
}
```

Con Expresiones Lambda:

```
public class EjerciciosLambda {
    public static void main(String[] args) {
        String cadena = "Hola";
        int multiplicador = 3;

        // Expresión lambda para multiplicar un string por un número
entero
        MultiplicarString multiplicarString = (str, n) -> {
            StringBuilder resultado = new StringBuilder();
            for (int i = 0; i < n; i++) {
                resultado.append(str);
            }
            return resultado.toString();
        };

        String resultadoMultiplicacion =
multiplicarString.multiplicarString(cadena, multiplicador);
        System.out.println("Resultado de la Multiplicación: " +
resultadoMultiplicacion);
    }

    interface MultiplicarString {
        String multiplicarString(String str, int n);
    }
}
```

EJERCICIOS CON EXPRESIÓN LAMBDA EN COLECCIONES: USO DE LA FUNCIÓN STREAM()

1. **Filtrado de una lista de números:**

   Dada una lista de números, filtra los números pares.

```
List<Integer> numeros = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

List<Integer> pares = numeros.stream()
                            .filter(num -> num % 2 == 0)
                            .collect(Collectors.toList());

System.out.println("Números pares: " + pares);
```

Explicación:

Este código realiza las siguientes operaciones utilizando **streams**:

1. **Creación de la lista de números:** Se crea una lista de números enteros utilizando el método estático **Arrays.asList()**, que convierte una matriz en una lista. En este caso, se pasa una serie de números del 1 al 10 como argumentos al método **asList()**, y se asigna el resultado a la lista numeros.

2. **Filtrado de números pares:** Se utiliza el método **stream()** de la lista numeros para obtener un stream de los elementos de la lista. Luego, se llama al método **filter()** del stream para filtrar solo los números pares. Esto se hace utilizando una expresión lambda **(num -> num % 2 == 0)** que verifica si un número es divisible por 2, es decir, si es par. La función **filter()** conservará solo los elementos que cumplan con esta condición.

3. **Recopilación de resultados:** Después de filtrar los números pares, se llama al método **collect()** para recopilar los resultados del stream filtrado. Se utiliza **Collectors.toList()** como argumento para **collect()** para recopilar los elementos filtrados en una nueva lista.

4. **Impresión de los números pares:** Finalmente, se imprime la lista de números pares utilizando **System.out.println()**. Se concatena la cadena **"Números pares: "** con la lista pares utilizando **el operador +**, lo que convierte automáticamente la lista en una representación de cadena.

2. **Transformación de una lista de strings:**

Dada una lista de strings, convierte cada string a mayúsculas.

```
List<String> palabras = Arrays.asList("hola", "mundo", "java",
"programación");

List<String> mayusculas = palabras.stream()
                                .map(String::toUpperCase)
                                .collect(Collectors.toList());

System.out.println("Palabras en mayúsculas: " + mayusculas);
```

Explicación:

Este código utiliza streams en Java para realizar las siguientes operaciones:

1. **Creación de la lista de palabras:** Se crea una lista de **strings** utilizando el método estático **Arrays.asList()**, que convierte una matriz en una lista. En este caso, se pasa una serie de palabras como argumentos al método **asList()** y se asigna el resultado a la lista palabras.

2. **Mapeo a mayúsculas:** Se utiliza el método **stream()** de la lista palabras para obtener un stream de los elementos de la lista. Luego, se llama al método **map()** del stream para transformar cada palabra a mayúsculas. En este caso, se utiliza una referencia de método **String::toUpperCase**, que es equivalente a una expresión lambda que llama al método **toUpperCase()** en cada elemento de la lista. Este método convierte cada palabra a mayúsculas.

3. **Recopilación de resultados:** Después de aplicar la transformación a mayúsculas, se llama al método **collect()** para recopilar los resultados del stream transformado. Se utiliza **Collectors.toList()** como argumento para **collect()** para recopilar los elementos transformados en una nueva lista.

4. **Impresión de las palabras en mayúsculas:** Finalmente, se imprime la lista de palabras en mayúsculas utilizando **System.out.println()**. Se concatena la cadena "Palabras en mayúsculas: " con la lista **mayúsculas** utilizando el **operador +**, lo que convierte automáticamente la lista en una representación de cadena.

3. **Suma de una lista de números:**

Dada una lista de números, calcula la suma de todos los elementos.

```
List<Integer> numeros = Arrays.asList(1, 2, 3, 4, 5);

int suma = numeros.stream()
                 .reduce(0, Integer::sum);

System.out.println("Suma de los números: " + suma);
```

Explicación:

Este código utiliza **streams** en Java para realizar la siguiente operación:

1. **Creación de la lista de números:** Se crea una lista de enteros utilizando el método estático **Arrays.asList()**, que convierte una serie de valores en una lista. En este caso, se pasa una secuencia de números como argumentos al método **asList()** y se asigna el resultado a la lista **numeros**.

2. **Reducción de la lista a la suma de sus elementos:** Se llama al método **stream()** de la lista **numeros** para obtener un **stream** de sus elementos. Luego, se llama al método reduce() del stream. El método **reduce()** toma dos parámetros: el valor inicial (en este caso, 0) y una función de reducción. La función de reducción es **Integer::sum**, que es una referencia de método que se refiere al método sum() de la clase **Integer**. Esta función se aplica de forma acumulativa a cada elemento del stream, reduciéndolos a un solo valor acumulado.

3. **Recopilación del resultado:** Después de aplicar la función de reducción, **reduce()** devuelve el resultado final de la operación de reducción. En este caso, devuelve la suma de todos los elementos de la lista.

4. **Impresión de la suma:** Finalmente, se imprime la suma de los números utilizando **System.out.println()**. Se concatena la cadena **"Suma de los números: "** con la suma calculada, convirtiéndola automáticamente en una representación de cadena.

4. **Ordenación de una lista de strings:**

Dada una lista de strings, ordénalos alfabéticamente.

```
List<String> palabras = Arrays.asList("manzana", "banana", "pera",
"naranja");

List<String> ordenadas = palabras.stream()
                                .sorted()
                                .collect(Collectors.toList());

System.out.println("Palabras ordenadas: " + ordenadas);
```

Explicación:

Este código utiliza **streams** en Java para ordenar una lista de palabras en orden alfabético. Aquí tienes una explicación paso a paso:

1. **Creación de la lista de palabras:** Se crea una lista de strings llamada palabras utilizando el método **Arrays.asList()**. Este método convierte los argumentos proporcionados en una lista.

2. **Obtención de un stream de la lista:** Se llama al método **stream()** en la lista palabras para obtener un stream de sus elementos.

3. **Ordenación de los elementos:** Se llama al método **sorted()** en el stream para ordenar los elementos. Por defecto, este método ordena los elementos en orden natural (alfabético en el caso de las cadenas).

4. **Recopilación del resultado:** Después de ordenar los elementos, se llama al método **collect()** en el stream para recopilar los elementos ordenados en una lista. Se utiliza **Collectors.toList()** para especificar que se debe crear una lista como resultado.

5. **Impresión de las palabras ordenadas:** Finalmente, se imprime la lista de palabras ordenadas utilizando **System.out.println()**. La lista ordenada se convierte automáticamente en una cadena al concatenarla con la cadena "Palabras ordenadas: ".

5. **Conteo de elementos en una lista:**

Dada una lista de strings, cuenta cuántas veces aparece cada palabra.

```
List<String> palabras = Arrays.asList("hola", "mundo", "hola",
"java", "mundo", "java", "java");

Map<String, Long> conteo = palabras.stream()
.collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()));

System.out.println("Conteo de palabras: " + conteo);
```

Explicación:

Este código cuenta la frecuencia de ocurrencia de cada palabra en una lista de palabras. Aquí tienes una explicación paso a paso:

1. **Creación de la lista de palabras:** Se crea una lista de strings llamada palabras utilizando el método **Arrays.asList().** Esta lista contiene varias palabras, algunas de las cuales se repiten.

2. **Obtención de un stream de la lista:** Se llama al método **stream()** en la lista palabras para obtener un stream de sus elementos.

3. **Agrupación y conteo de las palabras:** Se utiliza el método **collect()** en el stream para recopilar los elementos en un mapa. Para agrupar las palabras por su contenido y contar cuántas veces aparece cada una, se utiliza el método **Collectors.groupingBy()** con **Function.identity()** como clasificador, que simplemente devuelve cada elemento sin modificarlo, y **Collectors.counting()** como operación de reducción, que cuenta el número de elementos en cada grupo.

4. **Obtención del resultado:** El resultado de la operación de conteo se almacena en un mapa donde las claves son las palabras y los valores son las frecuencias de ocurrencia de cada palabra.

5. **Impresión del conteo de palabras:** Finalmente, se imprime el mapa de conteo de palabras utilizando **System.out.println().** La representación impresa del mapa muestra las palabras como claves y el número de ocurrencias como valores.

6. **Filtrado y mapeo de una lista de objetos:**

Dada una lista de objetos de tipo `Persona`, filtra las personas mayores de 18 años y luego mapea sus nombres.

```
List<Persona> personas = Arrays.asList(
    new Persona("Juan", 20),
    new Persona("María", 25),
    new Persona("Pedro", 18),
    new Persona("Ana", 30)
);

List<String> nombresMayores = personas.stream()
                                    .filter(persona ->
persona.getEdad() > 18)
                                    .map(Persona::getNombre)
                                    .collect(Collectors.toList());

System.out.println("Nombres de personas mayores de 18 años: " +
nombresMayores);
```

Explicación:

Este código utiliza streams en Java para filtrar personas mayores de 18 años de una lista de personas, extraer sus nombres y almacenarlos en una lista separada, y luego imprime esta lista de nombres.

1. **Creación de una lista de personas:** Se crea una lista de objetos de tipo Persona utilizando el método **Arrays.asList().** Cada objeto Persona tiene un nombre y una edad.

2. **Obtención de un stream de la lista de personas:** Se llama al método **stream()** en la lista de personas para obtener un stream de sus elementos.

3. **Filtrado de personas mayores de 18 años:** Se utiliza el método **filter()** en el stream para filtrar solo aquellas personas cuya edad sea mayor que 18.

4. **Mapeo de nombres:** Se aplica el método **map()** para obtener solo los nombres de las personas filtradas. El método **Persona::getNombre** se utiliza como referencia a método para extraer el nombre de cada objeto Persona.

5. **Recopilación de nombres en una lista:** Se utiliza el método **collect()** con **Collectors.toList()** para recopilar los nombres mapeados en una nueva lista.

6. **Impresión de los nombres:** Finalmente, se imprime la lista de nombres de personas mayores de 18 años utilizando **System.out.println().**

7. **Agrupación de elementos en una lista de objetos:**

Dada una lista de objetos de tipo `Producto`, agrupa los productos por categoría.

```
List<Producto> productos = Arrays.asList(
    new Producto("Laptop", "Electrónico"),
    new Producto("Mouse", "Accesorio"),
    new Producto("Teclado", "Accesorio"),
    new Producto("Smartphone", "Electrónico")
);

Map<String, List<Producto>> productosPorCategoria =
productos.stream()
.collect(Collectors.groupingBy(Producto::getCategoria));

//1. Si hemos creado el método toString() en la clase Producto:

System.out.println("Productos por categoría: " +
productosPorCategoria);

//2. Si no hemos creado un método toString() en la clase Producto:

productosAgrupados.forEach((categoria, listaProductos) -> {
    System.out.println("Categoría: " + categoria);

    //No podemos hacer un foreach con :: de listaProducto si no he-
    mos creado un toString() en la clase Producto.
    for(Producto person: listaProductos)
        System.out.println("Producto: " + person.getNombre() +
                           ", Categoría: " + person.getCategoria() +
                           ", Precio: " + person.getPrecio());
}
```

Explicación:

(Antes hay que crear la clase Producto con constructor y getters)

1. Se crea una lista de objetos **Producto** con algunos elementos.

2. Luego, se utiliza el método **stream()** para convertir la lista en un stream de elementos.

3. Se aplica el método **collect(Collectors.groupingBy(Producto::getCategoria))** al stream para agrupar los productos por su categoría. Esto devuelve un mapa donde las claves son las categorías y los valores son listas de productos pertenecientes a cada categoría.

4. Si la clase **Producto** tiene un método **toString()** definido que devuelve una representación de texto del objeto, se imprime el mapa directamente usando **System.out.println("Productos por categoría: " + productosPorCategoria);**.

5. Si no se ha definido un método **toString()** en la clase **Producto**, se itera sobre el mapa resultante. Para cada categoría, se imprime la categoría y luego se itera sobre la lista de productos correspondiente a esa categoría. Dentro del bucle interno, se imprime el nombre, la categoría y el precio de cada producto.

8. **Reducción de una lista de números:**

Dada una lista de números, encuentra el máximo valor.

```
List<Integer> numeros = Arrays.asList(3, 1, 7, 5, 2);

Optional<Integer> maximo = numeros.stream().reduce(Integer::max);

System.out.println("Máximo valor: " + maximo.orElse(null));
```

Explicación:

Este código encuentra el máximo valor en la **lista de números** y lo imprime. El uso de **Optional** garantiza una manejo seguro en caso de que la lista esté vacía.

1. Se crea una lista de números enteros llamada **numeros** con algunos elementos.
2. Luego, se utiliza el método **stream()** para convertir la lista en un **stream** de elementos.
3. Se aplica el método **reduce(Integer::max)** al stream para encontrar el valor máximo en la lista. Este método utiliza una función de referencia **Integer::max**, que compara dos números y devuelve el mayor de los dos.
4. El resultado se almacena en un objeto **Optional<Integer>**. **Optional** es una clase que representa un valor que puede ser opcionalmente presente o ausente, lo que evita posibles excepciones de puntero nulo.
5. Finalmente, se imprime el máximo valor encontrado utilizando **System.out.println("Máximo valor: " + maximo.orElse(null));**. Si el valor máximo está presente en el **Optional**, se imprime; de lo contrario, se imprime **null**.

9. **Combinación de dos listas:**

Dadas dos listas de strings, combina sus elementos en una nueva lista.

```
List<String> lista1 = Arrays.asList("a", "b", "c");

List<String> lista2 = Arrays.asList("x", "y", "z");

List<String> combinacion = Stream.concat(lista1.stream(),
lista2.stream()).collect(Collectors.toList());

System.out.println("Lista combinada: " + combinacion);
```

Explicación:

Este código combina los elementos de dos listas de strings en una sola lista y luego imprime esa lista combinada.

1. Se crean dos listas de strings llamadas **lista1 y lista2**, cada una con algunos elementos.

2. Luego, se utiliza el método **concat()** de **la clase Stream** para concatenar los elementos de ambas listas en un solo stream.

3. Después, se utiliza el método **collect(Collectors.toList())** para recolectar los elementos del stream en una nueva lista.

4. Finalmente, se imprime la lista combinada utilizando **System.out.println("Lista combinada: " + combinacion);**.

10. Filtrado y conteo condicional:

Dada una lista de strings, filtra las palabras que comienzan con una vocal y cuenta cuántas son.

```
List<String> palabras = Arrays.asList("java", "hola", "mundo",
"programación", "eclipse", "idea");

long cantidad = palabras.stream().filter(palabra -> "aeiou"
                        .contains(String.valueOf(palabra.charAt(0))))
                        .count();

System.out.println("Palabras que empiezan con vocal: " + cantidad);
```

Explicación:

Este código cuenta cuántas palabras en la lista empiezan con una vocal y muestra el resultado por consola.

1. Se crea una lista de strings llamada palabras con algunas palabras.

2. Luego, se utiliza el método **stream()** para convertir la lista en un stream de strings.

3. A continuación, se aplica el método **filter()** para filtrar solo aquellas palabras cuyo primer carácter es una vocal. Esto se hace utilizando una expresión lambda que toma cada palabra, obtiene su primer carácter con **charAt(0)**, luego convierte ese carácter en un string con **String.valueOf()** y finalmente verifica si ese string está contenido en la cadena **"aeiou"**, que representa las vocales en minúsculas.

4. Después, se llama al método **count()** para contar cuántas palabras cumplen con la condición del filtro.

5. Finalmente, se imprime la cantidad de palabras que comienzan con una vocal utilizando **System.out.println("Palabras que empiezan con vocal: " + cantidad);**.

11. **Operaciones con mapas:**

Dado un mapa de nombres y edades, filtra las personas menores de 30 años y calcula el promedio de sus edades.

```
Map<String, Integer> personas = new HashMap<>();
personas.put("Juan", 25);
personas.put("María", 30);
personas.put("Pedro", 28);
personas.put("Ana", 35);

Double promedioEdadesMenores30 = personas.entrySet().stream()
        .filter(entry -> entry.getValue() < 30)
        .mapToInt(Map.Entry::getValue)
        .average()
        .orElse(0.0);

System.out.println("Promedio de edades de personas menores de 30
años: " + promedioEdadesMenores30);
```

Explicación:

Este código calcula el promedio de las edades de las personas que tienen menos de 30 años y lo imprime por consola.

1. Se crea un **HashMap** llamado personas, donde las claves son los nombres de las personas y los valores son sus edades.

2. Se utilizan los métodos **put()** para agregar elementos al mapa.

3. Luego, se utiliza el método **entrySet()** para obtener un conjunto de entradas del mapa, donde cada entrada representa un par clave-valor.

4. Se convierte este conjunto en un **stream** utilizando el método **stream()**.

5. Se aplica el método **filter()** al stream para filtrar solo aquellas entradas donde el valor (la edad) es menor que 30. Esto se hace con una **expresión lambda** que verifica si el valor es menor que 30.

6. Después, se utiliza **mapToInt()** para mapear cada entrada al valor de **la edad (representado por Map.Entry::getValue).**

7. Luego se llama a **average()** para calcular el promedio de todas las edades menores de 30.

8. Si no hay edades menores de 30, se devuelve 0.0 utilizando **orElse(0.0).**

9. Finalmente, se imprime el promedio de edades de las personas menores de 30 años.

12. **Operaciones con conjuntos:**

Dados dos conjuntos de números, encuentra la intersección y la unión de los conjuntos.

```
Set<Integer> conjunto1 = new HashSet<>(Arrays.asList(1, 2, 3, 4, 5));
Set<Integer> conjunto2 = new HashSet<>(Arrays.asList(4, 5, 6, 7, 8));

// Intersección de conjuntos
Set<Integer> interseccion = new HashSet<>(conjunto1);
interseccion.retainAll(conjunto2);
System.out.println("Intersección de conjuntos: " + interseccion);

// Unión de conjuntos
Set<Integer> union = new HashSet<>(conjunto1);
union.addAll(conjunto2);
System.out.println("Unión de conjuntos: " + union);
```

Explicación:

Este código realiza dos operaciones comunes con conjuntos: **la intersección y la unión.**

1. **Intersección de conjuntos:**
   - Se crea un nuevo **HashSet** llamado **interseccion** inicializado con los elementos del **conjunto1.**
   - Luego se llama al método **retainAll()** en **interseccion**, pasando **conjunto2** como argumento. Este método modifica interseccion para retener solo los elementos que también están presentes en **conjunto2.**
   - Finalmente, se imprime la intersección resultante de los conjuntos.
2. **Unión de conjuntos:**
   - Se crea un nuevo **HashSet** llamado **union** inicializado con los elementos del conjunto1.
   - Luego se llama al método **addAll()** en **union**, pasando **conjunto2** como argumento. Este método agrega todos los elementos de **conjunto2** a **union**, asegurándose de que no haya elementos duplicados.
   - Finalmente, se imprime la unión resultante de los conjuntos.

13. **Reducción de una lista de objetos:**

Dada una lista de objetos de tipo `Empleado`, encuentra el empleado con el salario más alto.

```
List<Empleado> empleados = Arrays.asList(
        new Empleado("Juan", 2500),
        new Empleado("María", 3000),
        new Empleado("Pedro", 3500),
        new Empleado("Ana", 2800)
);

Optional<Empleado> empleadoSalarioMaximo = empleados.stream()
        .max(Comparator.comparing(Empleado::getSalario));

System.out.println("Empleado con el salario más alto: " +
empleadoSalarioMaximo.orElse(null));
```

Explicación:

Este código busca encontrar el empleado con el salario más alto en una lista de objetos Empleado.

1. **Creación de la lista de empleados:**
   - Se crea una lista llamada **empleados** que contiene objetos de la clase **Empleado**. Cada empleado tiene **un nombre** y **un salario**.
2. **Búsqueda del empleado con el salario más alto:**
   - Se llama al método **stream()** en la lista de empleados para obtener un flujo de elementos.
   - Luego, se utiliza el método **max()** para encontrar el elemento máximo en el flujo de empleados. Se proporciona un comparador que compara los empleados basándose en sus salarios utilizando el método **Comparator.comparing()**, que toma una referencia de método a **Empleado::getSalario**.
   - El resultado de **max()** es un **Optional<Empleado>**, que puede contener el empleado con el salario más alto o estar vacío si la lista de empleados está vacía.
   - Se utiliza **orElse(null)** para obtener el empleado con el salario más alto si está presente, o **null** si la lista de empleados está vacía.
3. **Impresión del resultado:**
   - Se imprime en la consola el resultado, que es el empleado con el salario más alto encontrado, o **null** si la lista de empleados está vacía.

14. **Agrupación y conteo en una lista de objetos:**

Dada una lista de objetos de tipo `Producto`, agrupa los productos por fabricante y cuenta cuántos productos tiene cada fabricante.

```
List<Producto> productos = Arrays.asList(
        new Producto("Laptop", "Asus"),
        new Producto("Smartphone", "Samsung"),
        new Producto("Impresora", "HP"),
        new Producto("Tablet", "Asus"),
        new Producto("Monitor", "Samsung"),
        new Producto("Teclado", "HP"),
        new Producto("Mouse", "Asus")
);

Map<String, Long> productosPorFabricante = productos.stream()
        .collect(Collectors.groupingBy(Producto::getFabricante,
Collectors.counting()));

System.out.println("Cantidad de productos por fabricante: " +
productosPorFabricante);
```

Explicación:

Este código agrupa los productos por fabricante y cuenta cuántos productos tiene cada fabricante en una lista de objetos Producto.

1. **Creación de la lista de productos:**
   - Se crea una lista llamada **productos** que contiene objetos de la clase **Producto**. Cada producto tiene un **nombre** y un **fabricante**.
2. **Agrupación de productos por fabricante:**
   - Se llama al método **stream()** en la lista de productos para obtener un flujo de elementos.
   - Se utiliza el método **collect()** junto con **Collectors.groupingBy()** para agrupar los productos por el atributo fabricante. Este método crea un mapa donde las claves son los fabricantes y los valores son listas de productos correspondientes a cada fabricante.
   - Se utiliza **Collectors.counting()** como segundo argumento de **groupingBy()** para contar los elementos en cada grupo. Esto devuelve un mapa donde las claves son los fabricantes y los valores son el recuento de productos correspondientes a cada fabricante.
3. **Impresión del resultado:**
   - Se imprime en la consola el resultado, que es un mapa que muestra la cantidad de productos por fabricante.

15. **Filtrado y clasificación en una lista de objetos:**

Dada una lista de objetos de tipo `Libro`, filtra los libros publicados después de cierto año y ordénalos por título.

```
List<Libro> libros = Arrays.asList(
   new Libro("1984", "George Orwell", 1949),
   new Libro("Cien años de soledad", "Gabriel García Márquez", 1967),
   new Libro("El gran Gatsby", "F. Scott Fitzgerald", 1925),
   new Libro("El señor de los anillos", "J.R.R. Tolkien", 1954),
   new Libro("El Hobbit", "J.R.R. Tolkien", 1937)
);

int añoPublicacion = 1950;

List<Libro> librosDespuesDeAño = libros.stream()
        .filter(libro -> libro.getAñoPublicacion() > añoPublicacion)
        .sorted(Comparator.comparing(Libro::getTitulo))
        .collect(Collectors.toList());

System.out.println("Libros publicados después de " + añoPublicacion +
": " + librosDespuesDeAño);
```

Explicación:

Este código filtra los libros que fueron publicados después de un año específico, los ordena por título y los almacena en una lista.

1. **Creación de la lista de libros:**
   - Se crea una lista llamada **libros** que contiene objetos de la clase **Libro**. Cada libro tiene un **título**, **un autor y un año de publicación**.
2. **Filtrado de los libros por año de publicación:**
   - Se llama al método **stream()** en la lista de libros para obtener un **flujo de elementos**.
   - Se utiliza el método **filter()** para filtrar los libros cuyo año de publicación es posterior al año proporcionado **(añoPublicacion)**.
3. **Ordenación de los libros por título:**
   - Se utiliza el método **sorted()** para ordenar los libros filtrados por título. Se proporciona un comparador generado con **Comparator.comparing()** que compara los libros por el título.
4. **Almacenamiento de los libros en una lista:**
   - Se llama al método **collect()** con **Collectors.toList()** para recopilar los elementos del flujo en una lista.
5. **Impresión del resultado:**
   - Se imprime en la consola el resultado, que es una lista de libros publicados después del año especificado, ordenados alfabéticamente por título.

16. Filtrado y sumarización de una lista de números:

```
List<Integer> numeros = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

IntSummaryStatistics stats = numeros.stream()
                                    .filter(num -> num % 2 == 0)
                                    .mapToInt(Integer::intValue)
                                    .summaryStatistics();

System.out.println("Estadísticas de números pares: " + stats);
```

Explicación:

Este código calcula estadísticas sobre los números pares de una lista dada.

1. **Creación de la lista de números:**
   - Se crea una lista llamada **numeros** que contiene números del 1 al 10.
2. **Cálculo de las estadísticas:**
   - Se llama al método **stream()** en la lista de números para obtener un flujo de elementos.
   - Se utiliza el método **filter()** para filtrar solo los números pares. La expresión lambda **num -> num % 2 == 0** verifica si un número es divisible por 2, es decir, si es par.
   - Se aplica **mapToInt()** para convertir el **flujo de Integer** a un **flujo de int**, ya que **summaryStatistics()** opera en tipos primitivos.
   - Se llama al método **summaryStatistics()** para obtener **un resumen estadístico** de los números pares. Este método devuelve un objeto de tipo **IntSummaryStatistics**, que contiene estadísticas como el recuento, la suma, el mínimo, el máximo y la media de los números en el flujo.
3. **Impresión del resultado:**
   - Se imprime en la consola el resultado, que son las estadísticas de los números pares en la lista.

17. Concatenación de strings de una lista:

```
List<String> palabras = Arrays.asList("Hola", "mundo", "desde",
"Java");
String resultado = palabras.stream()
                          .collect(Collectors.joining(", "));

System.out.println("Frase concatenada: " + resultado);
```

Explicación:

Este código concatena una lista de cadenas en una sola cadena separada por comas.

1. **Creación de la lista de palabras:**
   - Se crea una lista llamada **palabras** que contiene las palabras **"Hola", "mundo", "desde" y "Java".**
2. **Concatenación de las palabras:**
   - Se llama al método **stream()** en la lista de palabras para obtener un flujo de elementos.
   - Se utiliza el método **collect()** con **Collectors.joining(", ")** para concatenar las palabras. El parámetro ", " especifica que se debe usar una coma seguida de un espacio como delimitador entre las palabras.
3. **Almacenamiento del resultado:**
   - El resultado de la concatenación se asigna a la variable resultado.
4. **Impresión del resultado:**
   - Se imprime en la consola el resultado, que es la frase concatenada con todas las palabras separadas por comas y espacios.

18. Exclusión de elementos comunes entre dos listas:

```
List<String> lista1 = Arrays.asList("a", "b", "c", "d", "e");

List<String> lista2 = Arrays.asList("c", "d", "e", "f", "g");

List<String> exclusivos = lista1.stream()
                     .filter(elemento -> !lista2.contains(elemento))
                     .collect(Collectors.toList());

System.out.println("Elementos exclusivos de lista1: " + exclusivos);
```

Explicación:

Este código filtra los elementos exclusivos de lista1 en comparación con lista2.

1. **Creación de las listas:**
   - Se crean dos listas llamadas **lista1 y lista2**, cada una contiene una serie de cadenas.
2. **Filtrado de elementos exclusivos:**
   - Se llama al método **stream()** en lista1 para obtener un flujo de elementos.
   - Se utiliza el método **filter()** para filtrar solo los elementos que no están presentes en **lista2**. La condición **!lista2.contains(elemento)** filtra aquellos elementos de **lista1** que no están en **lista2**.
   - El resultado de este filtro se almacena en una nueva lista llamada **exclusivos**.
3. **Almacenamiento de los elementos exclusivos:**
   - Se utiliza **collect(Collectors.toList())** para convertir el flujo filtrado en una lista.
4. **Impresión del resultado:**
   - Se imprime en la consola la lista exclusivos, que contiene los elementos exclusivos de lista1.

19. Obtención del producto de todos los elementos de una lista de números:

```
List<Integer> numeros = Arrays.asList(1, 2, 3, 4, 5);

Optional<Integer> producto = numeros.stream()
                                  .reduce((a, b) -> a * b);

System.out.println("Producto de los números: " +
producto.orElse(null));
```

Explicación:

Este código calcula el producto de todos los números en la lista **numeros**.

1. **Creación de la lista:**
   - Se crea una lista llamada **numeros** que contiene una serie de números enteros.
2. **Cálculo del producto:**
   - Se llama al método **stream()** en **numeros** para obtener un flujo de elementos.
   - Se utiliza el método **reduce()** para calcular el producto de todos los elementos en el flujo. Dentro del **reduce()**, la expresión **lambda (a, b) -> a * b** toma dos parámetros **a y b** y devuelve su **producto**.
   - El resultado del producto se almacena en un objeto **Optional<Integer>** llamado **producto**.
3. **Obtención del resultado:**
   - Se utiliza **producto.orElse(null)** para obtener el valor del producto. Si el producto existe (es decir, si la lista **numeros** no está vacía), se devuelve el valor del **producto**. De lo contrario, se devuelve **null**.
4. **Impresión del resultado:**
   - Se imprime en la consola el producto de los números.

20. Filtrado y transformación de una lista de objetos:

```
List<Empleado> empleados = Arrays.asList(
    new Empleado("Juan", 1000),
    new Empleado("María", 1200),
    new Empleado("Pedro", 1500),
    new Empleado("Ana", 1100)
);

Map<String, Integer> salarioPorNombre = empleados.stream()
                                                .filter(empleado ->
empleado.getSalario() > 1000)
.collect(Collectors.toMap(Empleado::getNombre, Empleado::getSalario));

System.out.println("Salario de los empleados con salario mayor a 1000:
" + salarioPorNombre);
```

Explicación:

Este código filtra los empleados cuyo salario es mayor a 1000 y luego crea un mapa donde las claves son los nombres de estos empleados y los valores son sus salarios. Finalmente, imprime este mapa en la consola.

1. **Creación de la lista de empleados:**
   - Se crea una lista llamada **empleados** que contiene objetos de tipo **Empleado**. Cada empleado tiene un **nombre** y un **salario**.
2. **Filtrado de empleados con salario mayor a 1000:**
   - Se llama al método **stream()** en la lista empleados para obtener un flujo de elementos.
   - Se utiliza el método **filter()** para filtrar los empleados cuyo salario es mayor a 1000.
3. **Creación del mapa de salario por nombre:**
   - Se utiliza el método **collect()** para recopilar los elementos del flujo en un mapa.
   - En el método **collect()**, se utiliza **Collectors.toMap()** para crear un mapa donde la clave es el nombre del empleado (obtenido con **Empleado::getNombre**) y el valor es el salario del empleado (obtenido con **Empleado::getSalario**).
4. **Impresión del resultado:**
   - Se imprime en la consola el mapa **salarioPorNombre**, que contiene los nombres de los empleados con un salario mayor a 1000 y sus respectivos salarios.

21. **Ejemplo de clasificación de una lista de objetos por múltiples atributos:**

Supongamos que tenemos una lista de objetos `Persona` con atributos como nombre, edad y ciudad. Queremos ordenar esta lista primero por ciudad y luego por edad.

```
import java.util.*;
import java.util.stream.Collectors;

class Persona {

    private String nombre;
    private int edad;
    private String ciudad;

    public Persona(String nombre, int edad, String ciudad) {
        this.nombre = nombre;
        this.edad = edad;
        this.ciudad = ciudad;
    }

    public String getNombre() { return nombre; }

    public int getEdad() { return edad; }

    public String getCiudad() { return ciudad; }

    @Override
    public String toString() {
        return nombre + " - " + edad + " años - " + ciudad;
    }
}

public class Main {

    public static void main(String[] args) {
        List<Persona> personas = Arrays.asList(
                new Persona("Juan", 30, "Madrid"),
                new Persona("María", 25, "Barcelona"),
                new Persona("Pedro", 35, "Madrid"),
                new Persona("Ana", 28, "Barcelona")
        );

        List<Persona> personasOrdenadas = personas.stream()
                .sorted(Comparator.comparing(Persona::getCiudad)
                        .thenComparing(Persona::getEdad))
                .collect(Collectors.toList());

        personasOrdenadas.forEach(System.out::println);
    }
}
```

Explicación:

este código crea una lista de personas, las ordena primero por ciudad y luego por edad, y finalmente imprime las personas ordenadas en la consola.

1. **Definición de la clase Persona:**
   - Se define una clase **Persona** que tiene tres atributos: **nombre, edad y ciudad.**
   - Se proporciona un **constructor** para inicializar los atributos de la persona.
   - Se definen métodos para obtener el nombre, la edad y la ciudad de la persona.
   - Se sobrescribe el método **toString()** para proporcionar una representación de cadena de la persona en el formato **"nombre - edad años - ciudad".**
2. **Clase principal Main:**
   - Se define la clase principal **Main.**
   - En el método **main**, se crea una lista llamada **personas** que contiene objetos de tipo **Persona.** Cada persona tiene un **nombre, edad y ciudad.**
3. **Ordenamiento de la lista de personas:**
   - Se utiliza el método **stream()** en la lista personas para obtener un flujo de elementos.
   - Se llama al método **sorted()** para ordenar las personas. Se utiliza **Comparator.comparing()** para especificar el criterio de ordenamiento. En este caso, se ordena primero por ciudad y luego por edad utilizando **thenComparing().**
4. **Recopilación de los resultados en una lista:**
   - Se utiliza el método **collect()** para recopilar los elementos ordenados en una lista.
5. **Impresión de las personas ordenadas:**
   - Se utiliza el método **forEach()** para imprimir cada persona ordenada en la consola utilizando **System.out.println().**

22. **Ejemplo de agrupación de una lista de objetos por múltiples atributos:**

Supongamos que tenemos una lista de objetos `Producto` con atributos como categoría y precio. Queremos agrupar estos productos por categoría y, dentro de cada categoría, queremos que estén ordenados por precio de forma ascendente.

```
import java.util.*;
import java.util.stream.Collectors;

class Producto {

    private String nombre;
    private String categoria;
    private double precio;

    public Producto(String nombre, String categoria, double precio) {
        this.nombre = nombre;
        this.categoria = categoria;
        this.precio = precio;
    }

    public String getNombre() { return nombre; }

    public String getCategoria() { categoria; }

    public double getPrecio() { return precio; }

    @Override
    public String toString() {
        return nombre + " - " + categoria + " - $" + precio;
    }
}

public class Main {

    public static void main(String[] args) {

        List<Producto> productos = Arrays.asList(
                new Producto("Portátil", "Electrónica", 899.99),
                new Producto("Teléfono", "Electrónica", 599.99),
                new Producto("Camisa", "Ropa", 29.99),
                new Producto("Pantalón", "Ropa", 39.99),
                new Producto("TV", "Electrónica", 1499.99)
        );
        Map<String, List<Producto>> productosAgrupados;

        productosAgrupados = productos.stream()
                .sorted(Comparator.comparing(Producto::getPrecio))
                .collect(Collectors.groupingBy(
                                Producto::getCategoria,
                                TreeMap::new,
                                Collectors.toList()));

        productosAgrupados.forEach((categoria, listaProductos) -> {
            System.out.println("Categoría: " + categoria);
            listaProductos.forEach(System.out::println);
        });
    }
}
```

Explicación:

Este código crea una lista de productos, los ordena por precio, los agrupa por categoría y finalmente imprime los productos agrupados por categoría, mostrando cada categoría seguida de los productos que pertenecen a ella.

1. **Definición de la clase Producto:**

   - Se define una clase **Producto** que tiene tres atributos: **nombre, categoria y precio.**
   - Se proporciona un **constructor** para inicializar los atributos del producto.
   - Se definen métodos para obtener **el nombre, la categoría y el precio del producto.**
   - Se sobrescribe el método **toString()** para proporcionar una representación de cadena del producto en el formato **"nombre - categoria - $precio".**

2. **Clase principal Main:**

   - Se define la clase principal **Main.**
   - En el método **main,** se crea una lista llamada **productos** que contiene objetos de tipo **Producto.** Cada producto tiene un **nombre, una categoría y un precio.**

3. **Ordenamiento de la lista de productos por precio:**

   - Se utiliza el método **stream()** en la lista productos para obtener un flujo de elementos.
   - Se llama al método **sorted()** para ordenar los productos. Se utiliza **Comparator.comparing()** para especificar el criterio de ordenamiento. En este caso, se ordena por precio.

4. **Agrupación de productos por categoría:**

   - Se utiliza el método **collect()** para recopilar los elementos agrupados en un mapa.
   - Se utiliza **Collectors.groupingBy()** para agrupar los productos por categoría. Se especifica el método **Producto::getCategoria** como clasificador de la agrupación.
   - Se utiliza **TreeMap::new** como proveedor de mapa para garantizar que las claves estén ordenadas alfabéticamente.
   - Se utiliza **Collectors.toList()** para recopilar los productos de cada categoría en una lista.

5. **Impresión de los productos agrupados:**

   - Se utiliza el método **forEach()** para iterar sobre cada categoría y su lista de productos correspondiente.
   - Dentro del bucle, se imprime el nombre de la categoría y luego se imprime cada producto de esa categoría utilizando **System.out.println().**